

## READING SHEET

# React Router v6

Module 3 · Frontend: React &amp; Next.js · Day 31 · Week 7

## PROGRAM

CodeOps · Full Stack

## WORK TYPE

Revision + mini-project

## EST. TIME

2–3 hours

**Tip** How to use this sheet

This is your take-away for Day 31 — the day Addis Eats stops being one screen. It revises client-side routing, nested layouts, dynamic parameters and protected routes. Everything you built in week 1 still applies; it simply gets spread across real URLs. Keep the route table in front of you as you work, and load every route cold in the address bar rather than only clicking through.

## Part 1 · Routing Fundamentals

A single-page application loads one HTML file and never reloads. That means the browser cannot change pages for you — React Router does it instead, using the History API to change the address bar without a request, then matching the new path against your routes.

| TRADITIONAL SITE              | SINGLE-PAGE APPLICATION             |
|-------------------------------|-------------------------------------|
| Each link requests a new page | The URL changes, no request is made |
| The server picks the HTML     | The router picks the component      |
| Full reload, state is lost    | State survives the navigation       |
| One HTML file per screen      | One HTML file, many routes          |

### The three components

```
npm install react-router-dom
```

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/menu" element={<Menu />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</BrowserRouter>
```

App.jsx – BrowserRouter, Routes, Route

**BrowserRouter** goes once at the top and provides the routing context. **Routes** picks the single best match for the current URL. Each **Route** pairs a path with an element. Always include `path="*" —` without it, a mistyped URL renders a blank screen with no explanation.

### Link, not anchor

```

// navigating without a reload
<a href="/menu">Menu</a>      // full reload, state lost
<Link to="/menu">Menu</Link>  // the router handles it

<NavLink to="/menu"
  className={({ isActive }) => isActive ? "on" : ""}>Menu</NavLink>

```

### Warning A plain anchor breaks the application

It asks the server for a page that does not exist as a file, reloads everything and wipes the cart. Inside your own application, always use Link. Reserve the plain anchor for external addresses — a TeleBirr payment page, or Ethiopian Airlines.

### The Addis Eats route table

| PATH      | SCREEN                               |
|-----------|--------------------------------------|
| /         | Landing page with today's specials   |
| /menu     | The full menu with category filter   |
| /menu/:id | One dish, with description and price |
| /cart     | The current order and total in ETB   |
| /checkout | Delivery details — signed in only    |
| *         | Not found                            |

Design the URLs before writing components. The route table is the shape of your application: which screens exist, which take a parameter, and which require the person to be signed in.

## Part 2 · Nested Routes & Layouts

Most screens share a header, a navigation bar and a footer. Repeating that frame in every screen is duplication — and worse, every navigation rebuilds it, throwing away any state it held. Nested routes let a parent render the frame once and drop the matching child into a hole in the middle.

```

// Layout.jsx and the nested route tree

function Layout() {
  return (
    <>
      <Header /> <Nav />
      <Outlet />      // the matched child renders here
      <Footer />
    </>
  );
}

<Route path="/" element={<Layout />}>
  <Route path="menu" element={<Menu />} />
  <Route path="cart" element={<Cart />} />
</Route>

```

Navigating from /menu to /cart swaps only what is inside Outlet. The header is never rebuilt — which is exactly why the CartProvider from Day 30 keeps the order intact as the student moves

between screens. Wrap the router or the layout in it, above everything that reads the cart and outside anything that unmounts.

### Index routes and relative paths

```
<Route path="/" element={<Layout />}>                                no leading slash inside a parent
  <Route index      element={<Home />} />    // matches "/"
  <Route path="menu" element={<Menu />} />    // "/menu"
</Route>
```

| WRITTEN AS   | MEANS                                   |
|--------------|-----------------------------------------|
| index        | The parent's own path, and nothing more |
| path="menu"  | Relative — the parent path plus /menu   |
| path="/menu" | Absolute — ignores the parent entirely  |

#### Warning The leading slash matters

A child path beginning with / is absolute and escapes its parent, so the nesting silently stops working. Inside a nested route, leave the slash off and let the paths compose.

## Part 3 · Dynamic Routes & Params

One route can serve every dish on the menu. A path segment starting with a colon is a **parameter**: the router matches anything there and hands you the value, so /menu/kitfo and /menu/shiro both render the same component with different data.

```
<Route path="menu/:id" element={<DishDetail />} />                    one route, forty dishes

// DishDetail.jsx
const { id } = useParams();                                           // "kitfo"
const { data: dish, loading } = useFetch(`/api/dishes/${id}`);
```

useParams returns an object keyed by the names in the path, and every value is a **string** — even when it looks like a number. Note that putting the id into the useFetch URL also puts it in that hook's dependency array, so navigating from one dish to another refetches automatically with no extra code.

### Linking to a dynamic route

```
dishes.map(dish => (
  <Link key={dish.id} to={` /menu/${dish.id}`}>
    <Dish {...dish} />
  </Link>
))                                                                    DishList.jsx — every card is a link
```

The key prop is React's bookkeeping for the list; the id in the URL is what the next screen reads. They often come from the same field but serve entirely separate purposes. A slug such as doro-wat reads better than 17 — whichever you choose, the route and the data must agree.

### Query strings for shareable state

Menu.jsx – the category filter, in the URL

```
const [params, setParams] = useSearchParams();
const category = params.get("category") ?? "All";

setParams({ category: "Vegan" }); // /menu?category=Vegan
```

**Tip Which state belongs in the URL**

useSearchParams works like useState, but the value lives in the query string — so filters, sort order, page number and search terms become shareable and survive a refresh. A URL is public and permanent, though: keep cart contents, form drafts and anything private in ordinary state.

**Two kinds of missing**

A path nobody claimed is caught by the \* route. But a valid path with an id that does not exist still *matches* — so the component itself has to notice and say so. It is the Day 29 pattern with one more empty case.

DishDetail.jsx – a real id that finds nothing

```
if (loading) return <Spinner />;
if (error)   return <ErrorNote message={error} />;
if (!dish)  return <NotFound message={`No dish called ${id}`} />;
return <DishCard {...dish} />;
```

**Part 4 · Navigation & Protected Routes**

Use **Link** when the person chooses to go somewhere. Use **useNavigate** when your code decides — after a form submits, a payment clears or a session expires. Use the **Navigate** component when you need to redirect while rendering.

Checkout.jsx – useNavigate

```
const navigate = useNavigate();

async function placeOrder() {
  const order = await submit(form);
  navigate(`/orders/${order.id}`); // go to the receipt
}

navigate(-1); // back one step
navigate("/menu", { replace: true }); // no new history entry
```

replace: true replaces the current history entry instead of adding one. After a completed order that stops the back button returning to the checkout form — and after a redirect it stops the person bouncing straight back to the page that just rejected them.

**Redirecting during render**

the Navigate component

```
// wrong: navigating while rendering is a side effect
if (!user) navigate("/login");

// right: render a redirect instead
if (!user) return <Navigate to="/login" replace />;
```

Calling navigate during render is exactly the side effect Day 29 warned about. Navigate is a component — returning it is an ordinary render that happens to change the URL.

**A protected route**

RequireAuth.jsx — a guard component

```
function RequireAuth({ children }) {
  const { user, loading } = useContext(AuthContext);
  const location = useLocation();

  if (loading) return <Spinner />;
  if (!user) return (
    <Navigate to="/login" replace state={{ from: location }} />;
  );
  return children;
}
```

**Warning Check loading before user**

While the session is still being read, user is null. Redirect on that and a signed-in student is thrown back to the login screen on every refresh. Wait for loading to finish, then decide.

Wrap the screen with it — `element=<RequireAuth><Checkout /></RequireAuth>`. Passing `state={{ from: location }}` remembers where they were going, so the login screen can send them back:

Login.jsx — return them where they were

```
const from = location.state?.from?.pathname ?? "/menu";
await login(phone);
navigate(from, { replace: true }); // back to /checkout
```

## Assigned Reading

### Required

1. [reactrouter.com — Tutorial](#) What / why: the official walkthrough, building a routed application end to end.
2. [reactrouter.com — Route](#) What / why: the Route API in full — index routes, nesting, relative paths and the splat route.

### Optional — go deeper

- [useSearchParams](#) — reading and writing the query string as if it were state.
- [useNavigate](#) — programmatic navigation, relative paths and the replace option.

## Exercises

Work these in order and commit as you go. Together they route your week 1 project.

1. Install react-router-dom and wrap the application in BrowserRouter with three routes.
2. Replace every anchor in your navigation with Link, and the active tab with NavLink.
3. Build a Layout with header, nav, Outlet and footer, and nest the screens inside it.
4. Add an index route for the landing page and a "\*" route rendering NotFound.
5. Add menu/:id, read the id with useParams, and link each dish card to its own page.
6. Move the category filter into the query string with useSearchParams.
7. Write RequireAuth, guard /checkout, and return the person there after signing in.

## Mini-Project · Addis Eats, Routed

### Tip What you build

Split your week 1 application across real screens: a landing page, a menu with a shareable filter, a page for every dish, a cart that survives every navigation, and a checkout that cannot be reached without signing in. One layout renders the frame; the router decides what fills it.

### Requirements

- A nested route table with a Layout parent, an index route and a "\*" catch-all.
- Navigation built from Link and NavLink, with the current screen visibly highlighted.
- A dynamic /menu/:id route reading the parameter with useParams.
- The category filter stored in the query string, so /menu?category=Vegan can be shared.
- The cart provider mounted above the router so the order survives navigation.
- A RequireAuth guard on /checkout that waits for loading and remembers the destination.
- Every route loading correctly when typed straight into the address bar.

### A worked start

```
starter – the route tree

<CartProvider>
  <BrowserRouter>
    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<Home />} />
      <Route path="menu" element={<Menu />} />
      <Route path="menu/:id" element={<DishDetail />} />
      <Route path="cart" element={<Cart />} />
      <Route path="checkout" element={
        <RequireAuth><Checkout /></RequireAuth> />
      } />
      <Route path="*" element={<NotFound />} />
    </Routes>
  </BrowserRouter>
</CartProvider>
```

### What to submit

Your GitHub repository with App.jsx, Layout.jsx, DishDetail.jsx and RequireAuth.jsx, plus a README listing every route.

### Check yourself

- Does adding a dish to the cart, then visiting /cart, still show the order?
- Does the header keep its state when you navigate between screens?
- Does /menu?category=Vegan show the filtered menu when opened in a new tab?
- Does /menu/not-a-dish say the dish was not found rather than crashing?
- Does opening /checkout while signed out send you to login, then back afterwards?
- Does refreshing while signed in leave you on the page rather than at the login screen?